00_python_crash_course

October 2, 2026

1 Uvod

- Guido van Rossum, 1991.g.
- Benevolent Dictator for Life (dobrovoljno se povukao 2018.; danas jezikom upravlja Steering Council)
- Monty Python's Flying Circus
- dinamički programski jezik
- success stories: https://www.python.org/about/success/
- instalacija: https://www.python.org (koristimo Python 3)

[ ]:
```
import antigravity
```

[ ]:
```
2+2
```

[ ]:
```
a = 4
```

[ ]:
```
a = "neki string"
```

- dinamički jezik
- multiparadigmatski jezik
  - proceduralan
  - objektno orijentiran jezik
  - funkcionalni jezik
- interaktivan
- portabilan
- open source
- brza krivulja učenja, lak za početnike

[ ]:
```
import this
```

[ ]:
```
print("Hello world!")
```

2 Organizacija i pokretanje programa

2.1 Interaktivno zadavanje naredbi, naredbu po naredbu.

Pogodno za testiranje dijelova koda.

[ ]:
```
a = 13
```

[ ]:
```
b = "a"
```

[ ]:
```
a*b
```

2.2 Pisanje skripti i pokretanje s python interpreterom.

[ ]:
```
a = 13
b = 'a'
print(a * b)
```

Spremiti sa ekstenzijom `.py`, naprimjer `test01.py`. Pokreniti iz komandne linije: `python test01.py` (ili `python3 test01.py`).

2.3 Pisanje izvršnih skripti (Linux/macOS) — *za samostalno čitanje*

[ ]:
```
#!/usr/bin/python3
a = 13
b = 'a'
print(a * b)
```

Spremiti sa ekstenzijom `.py`, naprimjer `test02.py`. Prva linija je direktiva shellu koji će interpreter koristiti za pokretanje skripte. Kombinacija znakova `#!` zove se "hash-bang" ili "she-bang".

Da bi se skripta mogla izvršiti, potrebno joj je dati pravo izvršavanja (executable).

```
chmod a+x test02.py
```

Sada je možemo pokreniti kao izvršni program bez navođenja interpretera.

```
./test02.py
```

2.4 Korištenje IDE programa (Visual Studio Code, PyCharm, Spyder, Jupyter Notebook/JupyterLab, ...).

[ ]:
```
def main():
    print('Hello world')

if __name__ == '__main__':
    main()
```

3 Leksička struktura Pythona

- Komentari počinju znakom #

[ ]:
```
# ovo je komentar
```

- Imena varijabli sadrže alfanumeričke znakove i znak podvlačenja (*underscore*), znamenka ne smije biti na prvom mjestu

[ ]:
```
a_1 = 4   # ok
```

[ ]:
```
#1_a = 5    #nije ok
```

- Blokovi su određeni uvlačenjem retka. Preporuka 4 *spacea*.

[ ]:
```
for i in range(10,20,3):
    print(i, end=' ')
```

- Operatori: `+, -, ~, *, **, /, //, %, <<, >>, &, |, ^, and, or, not, in, not in, is, is not, <, >, !=, ==, <=, >=`
- https://docs.python.org/3/reference/expressions.html#operator-precedence

[ ]:
```
for i in range(10):
    print(i, i**2, i**3)
```

PEP 8 *Style Guide for Python Code* https://peps.python.org/pep-0008/

3.1 Significant whitespace

ne miješati `<tab>` i `space` (iako većina IDE programa konvertira `<tab>` u `space`)

[ ]:
```
for i in range(5):
    print(i, '. redak', end = ' ')
    for j in range(6):
        print('(', i, ',', j, ')', end = ' ')
    print('\n')
```

4 Objekti i tipovi

4.1 Sve je objekt!

Može se pridružiti varijabli ili proslijediti funkciji kao argument. * Stringovi su objekti. Funkcije su objekti. Moduli su objekti. Tipovi su objekti. * Svaki objekt ima svojstva: * identifikator `id(obj)`, * tip `type(obj)`, * vrijednost.
* Identifikator i tip su nepromjenjivi. * Vrijednost objekta može biti promjenjiva (*mutable*) ili nepromjenjiva (*immutable*) * Svaki objekt može imati: * nula ili više metoda, * nula ili više imena. * Imena su dio prostora imena (*namespace*).

5 Imena

Ime se može pridružiti bilo kojem objektu.

[ ]:
```
a = 10
b = "tralala"
c = {2:'a', 3:4}
d = [a,b,c]
```

I funkcija je objekt.

[ ]:
```
def fibonacci(k):
    a, b = 1, 1
    i = 2
    while i <= k:
        a, b = b, a + b
        i += 1
    return a

f = fibonacci
print(f(5))
```

5.1 And now for something completely different

Umjesto varijabli Python ima imena i veze (*bindings*).

[ ]:
```
lst = [1, 2, 3]
print(lst, id(lst))
lst2 = lst          # novo ime za ISTI objekt, nema kopiranja
lst2.append(4)
print(lst, id(lst))
print(lst2, id(lst2))
```

6 Tipovi

Python je jezik sa dinamičkim tipovima. Nema deklaracije varijabli, tip varijable se određuje prema vrijednosti koju dobiva. Tokom izvođenja programa, tip varijable se može mijenjati.

[ ]:
```
a = "Hello, world!"
print(type(a))

a = 10
print(type(a))
```

- Numerički tipovi `int`, `float`, `complex` https://docs.python.org/3/library/stdtypes.html#numeric-types-int-float-complex

[ ]:
```
a = 12787948962332423432467678164712848564584596890893084860983098846354645
print(a, type(a))
```

[ ]:
```
a = 12.787948962332423432467678164712848655464567657
print(a, type(a))
```

- `None`, singleton objekt tipa `NoneType`.
  - Postoji samo jedan `None` objekt tj. sve varijable kojima je pridružen `None` pokazuju na isti objekt.

[ ]:
```
a = None
print(a, id(a), type(a))
```

```
b = None
print(b, id(b), type(b))
```

- Boolean `True`, `False`
  - Svaki objekt ima boolean vrijednost.
  - Sljedeći objekti imaju vrijednost `False`: None, False, 0, prazne kolekcije `""`, `[]`,`()`,`{}` i objekti klasa koji imaju metodu `__bool__` ili metodu `__len__` koja vraća `False` ili nulu.
  - Svi ostali imaju vrijednost `True`

[ ]:
```
print(a == b)
print(a is b)      # postoji samo jedan None objekt
```

- Sekvence
  - Stringovi `str`
  - Liste `list`
  - n-torke `tuple`
  - virtualne sekvence `range`
- Mapiranje
  - Rječnici (*dictionary*) `dict`
- Skupovi `set`, `frozenset` (za multiskup koristi se `collections.Counter`)
- Ostali tipovi: files, funkcije/metode, moduli, tipovi/klase, iteratori/generatori

6.1 Vrste tipova: mutabilni i nemutabilni tipovi

Dvije su vrste tipova - promjenjivi (mutabilni) - `list`, `dict`, `set` - nepromjenjivi (nemutabilni) - `str`, `tuple`, numerički tipovi, `frozenset`

String je nemutabilan, tj. ne može se promijeniti.

[ ]:
```
s = "Python crash course"
print(s, id(s))
s += ' 2026'
print(s, id(s))
```

Vidimo da se `id` varijable promijenio, odnosno kreiran je novi string.

[ ]:
```
#s[1] = 'a'     # TypeError: string je nemutabilan
```

Pridružimo li vrijednost string `s` varijabli `t`, ona će pokazivati na isti string.

[ ]:
```
t = s
print(s, id(s))
print(t, id(t))
```

[ ]:
```
t = "Novi string"
print(s, id(s))
print(t, id(t))
```

Pogledajmo sada primjer mutabilnog tipa (liste).

[ ]:
```
lst = [1, 2, 3, 4]
print(lst, id(lst))
lst.append(3+2)
print(lst, id(lst))
```

U ovom slučaju `id` se nije promijenio.

Unutar nepromjenjivog objekta možemo mijenjati promjenjive objekte.

[ ]:
```
a = (1, 2, ['a'])
a[2].append(3)
print(a)
```

[ ]:
```
a[2][1]
```

7 Naredbe

Python strogo odvaja naredbe i izraze. Naprimjer, pridjeljivanje (*assignment*) je naredba, nije izraz (iznimka je operator `:=`, uveden u Pythonu 3.8).

7.1 Naredba `pass`

- ne radi ništa

7.2 Naredba grananja `if`

[ ]:
```
n = int(input("Unesi n: "))

if n < 0:
    print('negativan')
elif n > 0:
    print('pozitivan')
else:
    print('nula')
```

7.3 Naredbe iteracije `while`

```
while izraz:
    naredba_1
    ...
    naredba_n
else:
    naredba__1
    ...
    naredba__n
```

- ako je izraz istinit, izvršavaju se naredbe unutar `while` bloka, te se proces ponavlja
- kad je izraz neistinit, izvršavaju se naredbe unutar `else` bloka
- ako iz `while` izađemo sa `break` ili exceptionom, `else` blok se ne izvršava

[ ]:
```
# najveci zajednicki djelitelj
a = int(input("a = "))
b = int(input("b = "))
while b != 0:
    a, b = b, a % b
else:
    print("petlja je zavrsila bez break")

print("NZD je", a)
```

[ ]:
```
# pogadjanje brojeva
n = 100

guess = int(input('pogodi broj : '))

while guess != n:
    if guess < n:
        print('broj je veci, pokusaj ponovo')
        guess = int(input('pogodi broj : '))

    elif guess > n:
        print('broj je manji, pokusaj ponovo')
        guess = int(input('pogodi broj : '))
print('bravo!')
```

7.4 Naredba iteracije `for`

`for` iterira po sekvenci

```
for <variable> in <sequence>:
    <statements>
else:
    <statements>
```

[ ]:
```
lang = ["hr", "fr", "en"]
for x in lang:
    print(x, end=' ')
print()

for x in range(1, 11):
    print(x, end=' ')
```

7.5 Naredba `break`

[ ]:
```
for n in range(20, 50):
    if n % 7 == 0:
        print('prvi visekratnik od 7:', n)
        break
```

7.6 Naredba `continue`

```
[ ]: for n in range(10):
         if n % 3 == 0:
             continue          # preskoci ostatak tijela petlje
         print(n, end=' ')
```

7.7 Naredba `with`

```
[ ]: # pisanje u datoteku
     with open('primjer.txt', 'w') as f:
         f.write('prvi redak\n')
         f.write('drugi redak\n')
         f.write('treci redak\n')

     # citanje iz datoteke
     with open('primjer.txt') as f:
         data = f.read()
     print(data)
```

8 Sekvence

8.1 Sekvence općenito

- uređeni container koji podržava random access pomoću indeksa
- podržava barem:
  - iteraciju
  - pristup dijelovima sekvence
  - računanje duljine
- ugrađene sekvence podržavaju:
  - slicing
  - ponavljanje (osim `range`)
  - spajanje (osim `range`)
- ugrađene sekvence su
  - stringovi (`str`)
  - liste (`list`)
  - uređene n-torke (`tuple`)
  - virtualne sekvence (`range`)

8.1.1 Operacije nad sekvencama

```
[ ]: lst = [1,2,3,6]   #lista
     str1 = "Aaaa"    #string
     str2 = 'Bbbb'
     t = (1, 2, 'a')  #tuple
```

- duljina sekvence: `len(seq)`
- iteracija: for element in seq: ...

- indeksiranje `seq[i]`
- negativno indeksiranje `seq[-i]`
- za mutabilne sekvence promjenu elemenata `seq[i] = value`
- odsječak `seq[i1:i2]` vraća podsekvencu koja sadrži elemente između `i1` i `i2` indeksa ne uključujući element na mjestu `i2`
- ponavljanje: `seq * n`
- konkatenacija `seq1 + seq2`
- linearno pretraživanje u sekvenci `element in seq` `element not in seq`

- `min(seq)`
- `max(seq)`
- `sum(seq)`

8.1.2 Slice notacija

'p'	'y'	't'	'h'	'o'	'n'
0	1	2	3	4	5
-6	-5	-4	-3	-2	-1

`seq` je lista, string ili tuple * `seq[low:high:step]` * `seq[:]`, `seq[::]` * `seq[low:]` * `seq[:high]` * `seq[low:high]` * `seq[::step]` * `seq[low::step]` * `seq[:high:step]`

8.1.3 Slice assignment

- Samo za mutabilne sekvence.

[ ]:
```
# using a slice
lst = [1,2,3,4,5,6]
lst[2:3] = [7,8,9,10]
print(lst)
```

8.1.4 Kopiranje sekvence

[ ]:
```
lst = [1,2,3]
clst = lst[:]
print(clst)

# kopiranje unatrag
clst = lst[::-1]
print(clst)
```

8.1.5 Brisanje elemenata iz sekvence

[ ]:
```
lst = [1, 2, 3, 4, 5, 6]
del lst[2]        # brise element na indeksu 2
print(lst)
del lst[0:2]      # brise odsjecak
```

```
print(lst)
```

Isti efekt postiže se *slice assignmentom* prazne liste:

[ ]:
```
lst = [1, 2, 3, 4, 5, 6]
lst[0:2] = []
print(lst)
```

8.2 Stringovi

- String je sekvenca znakova.
- Navodi se unutar dvostrukih ili jednostrukih navodnika.
- `"Flying circus"`
- `'Flying circus'`
- `"Monty Python's Flying Circus"`
- `'Knights Who Say "Ni!"'`

8.2.1 String - metode

- String je sekvenca pa ima iste metode kao i druge sekvence.
- String ima i svoje metode.
- String je **nemutabilan**, operacije koje "mijenjaju" string zapravo vraćaju novi string (npr. `upper`, `replace`).

[ ]:
```
a = "Python"
a.upper()
print(a)
```

[ ]:
```
a = a.upper()
print(a)
```

- za jedan karakter (string duljine 1)
  - `ord` konvertira karakter u `int`
  - `chr` konvertira integer u karakter
- neke operacije nad stringovima
  - `str.split()`
  - `separator.join(str)`
  - `s in str`
  - regularni izrazi

[ ]:
```
text = "bla  ble blu"
words = text.split()
joined = '*'.join(words)
print(joined)
```

8.2.2 Operacije nad stringovima

```
[ ]: a = ' one two three '
```

```
[ ]: a * 3
```

```
[ ]: a
```

```
[ ]: 'one' in a
```

```
[ ]: a[1:4] + a[-6:-1]
```

8.2.3 Metode nad stringovima

- `s.split([separator])` - dijeli string u listu podstringova
- `s.join(seq)` - spaja elemente sekvence stringova, sa `s` kao separatorom
- `strip`, `lstrip`, `rstrip` - skida *whitespace* (ili zadane znakove) s početka i/ili kraja
- `s.replace(old, new)` - kreira kopiju stringa u kojoj je podstring `old` zamijenjen s podstringom `new`

Prije izvršavanja predvidjeti rezultate sljedećih naredbi.

```
[ ]: a = " one * two*"
```

```
[ ]: a.isalpha()     # True ako su svi znakovi slova
```

```
[ ]: a = a.strip('o*')
```

```
[ ]: a
```

```
[ ]: lst = a.split(' ')
```

```
[ ]: lst
```

```
[ ]: lst[0].isalpha()
```

```
[ ]: joined = '**'.join(lst)
```

```
[ ]: joined
```

```
[ ]: joined = joined.replace('*', '')
```

```
[ ]: joined
```

8.2.4 Formatiranje stringa

f-stringovi (preporučeni način): izraz se piše unutar `{}`, a iza `:` format kao u `printf`-u.

Stariji način je `format % argument` (formati kao u `printf`-u u C-u) i susreće se u starijem kodu.

[ ]:
```
maxt = 30.5
mint = 10.25
print(f'maksimum je {maxt:4.2f}, minimum {mint:6.3f}')
print('maksimum je %4.2f, minimum %6.3f' % (maxt, mint))   # stari nacin
```

8.3 Liste

- random-access container, nije linkana lista (za linkanu listu koristiti `collections.deque`)
- mutabilna sekvenca
- kreira se sa uglatim zagradama `[...]`

[ ]:
```
lst = []
for i in range(5):
    lst.append(i ** 2)
print(lst)
```

List comprehension

[ ]:
```
lst = [i ** 2 for i in range(5)]
print(lst)
```

8.3.1 Operacije nad listama

Prije izvršavanja predvidjeti rezultate sljedećih naredbi.

[ ]:
```
lst[-2]
```

[ ]:
```
lst[1:3]
```

[ ]:
```
lst[::-1]
```

[ ]:
```
del lst[0]
lst
```

[ ]:
```
lst + [2]
```

[ ]:
```
lst
```

[ ]:
```
del lst[-2:]
lst
```

[ ]:
```
lst * 2
```

8.3.2 Metode liste

- `lst.append(el)` - dodaje element na kraj liste
- `lst.count(el)` - broji koliko puta se element pojavljuje u listi
- `lst.extend(seq)` - dodaje elemente iz sekvence u listu
- `lst.index(el)` - vraća index prvog pojavljivanja elementa u listi

- `lst.insert(pos, el)` - ubacuje element na određeno mjesto u listi
- `lst.pop(i=-1)` - uklanja element s pozicije `i` (zadano: zadnji) i vraća ga
- `lst.remove(el)` - briše element iz liste (prvo pojavljivanje)
- `lst.reverse()` - preokreće listu
- `lst.sort(key=None, reverse=False)` - sortira listu *in place*, vraća `None`; `key` je funkcija po kojoj se uspoređuje

8.3.3 Funkcije nad listama

- `len(lst)` - vraća duljinu liste
- `max(lst)` - vraća element koji ima najveću vrijednost
- `min(lst)` - vraća element koji ima najmanju vrijednost
- `list(seq)` - konvertira sekvencu u listu
- `sorted(seq)` - vraća novu sortiranu listu (bilo koja iterabilna kolekcija na ulazu)

8.3.4 Sortiranje po ključu (`key`)

Parametar `key` je funkcija koja za svaki element vraća vrijednost po kojoj se sortira. Vrlo često je to `lambda`.

[ ]:
```
words = ['zebra', 'mi', 'avion', 'kuca']
print(sorted(words))                # abecedno
print(sorted(words, key=len))       # po duljini
print(sorted(words, key=len, reverse=True))
```

[ ]:
```
# lista n-torki (ime, bodovi) - sortiranje po drugom elementu
results = [('Ana', 92), ('Marko', 45), ('Iva', 67)]
print(sorted(results, key=lambda r: r[1]))

# min i max takodjer primaju key
print(max(results, key=lambda r: r[1]))
```

8.4 Tuple

- **nemutabilne** sekvence
- najčešće za grupiranje manjeg broja elemenata (npr. više povratnih vrijednosti iz funkcije)
- ako se sastoji samo od jednog elementa, nakon elementa se piše zarez (`(5,)`)
- prazni tuple se kreira sa `()`
- kreira se pomoću operatora zarez, a mogu doći i okrugle zagrade

Prije izvršavanja predvidjeti rezultate sljedećih naredbi.

[ ]:
```
1, 2, 'aa'
```

[ ]:
```
(1,2,[2,'a'])
```

[ ]:
```
t = (1,2,[2,'a'])
```

[ ]:
```
# t[0] = 3 # greska, tuple je nemutabilan tip
```

[ ]:
```
t[2][0] = 4    # ispravno, t[2] je lista tj. mutabilni tip
```

[ ]:
```
t
```

8.5 Raspakiravanje, `enumerate` i `zip`

Raspakiravanje (*unpacking*): elementi sekvence dodijele se u više imena odjednom.

[ ]:
```
point = (3, 4)
x, y = point
print(x, y)

a, b = 1, 2
a, b = b, a          # zamjena vrijednosti bez pomocne varijable
print(a, b)

first, *rest = [10, 20, 30, 40]
print(first, rest)
```

[ ]:
```
# raspakiravanje u petlji
pairs = [(1, 'a'), (2, 'b'), (3, 'c')]
for num, letter in pairs:
    print(num, letter)
```

`enumerate` daje indeks i element zajedno - umjesto `for i in range(len(lst))`.

[ ]:
```
colors = ['crvena', 'zelena', 'plava']
for i, c in enumerate(colors):
    print(i, c)
```

`zip` spaja više sekvenci element po element.

[ ]:
```
names = ['Ana', 'Marko', 'Iva']
points = [92, 45, 67]
for name, p in zip(names, points):
    print(f'{name}: {p}')

# dictionary iz dvije liste
d = dict(zip(names, points))
print(d)
```

9 Ostali containeri

9.1 Dictionary

- `mutabilne` data strukture koje mapiraju `key` i `value`
- implementirani kao hash tabela, `key` mora biti *hashable* tj. nemutabilan
- operacije nad `dict`:
  - `d[key]` - dohvaćanje elementa s ključem `key`

  - `d[key] = value` - pridjeljivanje
  - iteracija po ključu i vrijednostima
  - `len(d)` - duljina dictionaryja
- kreiranje
  - `{k1 : v1, k2 : v2, ...}`
  - pomoću `dict()` : `d=dict(k1 = v1, k2 = v2, ...)`

Prije izvršavanja predvidjeti rezultate sljedećih naredbi.

[ ]:
```
d = {'ana' : 23, 'banana' : 25}
print(d['banana'])
```

[ ]:
```
d['banana'] += 3
print(d['banana'])
```

[ ]:
```
d['nana'] = 4

del d['banana']
```

[ ]:
```
for i in d:
    print(i)
```

9.1.1 `dict` metode

- `d.keys()`, `d.values()`, `d.items()` vraćaju view objekte
- za listu `list(d.keys())`

[ ]:
```
for i in d.keys():
    print(i)
```

[ ]:
```
for val in d.values():
    print(val)
```

[ ]:
```
for key, val in d.items():
    print(key, val)
```

- `d.update(d2)` dodaje parove key-value iz `d2`
- `d.copy()` vraća kopiju dictionaryja (shallow copy)
- `copy.deepcopy(d)` vraća duboku kopiju (deep copy) — funkcija iz modula `copy`, ne metoda rječnika
- `d.clear()` briše sve elemente u `d`

[ ]:
```
d1 = dict(a = [1,2,3], b = 10, c = '110')
d2 = {'c' : 5}
print('d1 = ', d1)
print('d2 = ', d2)


d1.update(d2)
```

```
d2.clear()
print('d1 = ', d1)
print('d2 = ', d2)
```

[ ]:
```
d1['a'] += [4,5]
print(d1)
```

[ ]:
```
d2 = d1.copy()
print('d1 = ', d1)
print('d2 = ', d2)

d2['a'].append(10)
print('d1 = ', d1)
print('d2 = ', d2)
```

[ ]:
```
import copy
d2 = copy.deepcopy(d1)
print('d1 = ', d1)
print('d2 = ', d2)

d2['a'].append(20)
print('d1 = ', d1)
print('d2 = ', d2)
```

9.1.2 Vrijednosti rječnika mogu biti kolekcije

Često je vrijednost lista ili skup, npr. grupiranje elemenata po nekom svojstvu.

[ ]:
```
words = ['ana', 'banana', 'avion', 'breskva', 'cesta']
groups = {}
for w in words:
    first = w[0]
    if first not in groups:
        groups[first] = []
    groups[first].append(w)
print(groups)
```

[ ]:
```
# isto pomocu setdefault
groups = {}
for w in words:
    groups.setdefault(w[0], []).append(w)
print(groups)

# iteracija po parovima kljuc - lista
for key, lst in groups.items():
    print(key, '->', len(lst), 'rijeci')
```

Dictionary comprehension

[ ]:
```python
squares = {i: i**2 for i in range(6)}
print(squares)

# kljuc s najvecom vrijednoscu
scores = {'Ana': 92, 'Marko': 45, 'Iva': 67}
print(max(scores, key=scores.get))
```

9.1.3 defaultdict

- klasa izvedena iz `dict`
- omogućava da ako mijenjamo vrijednost ključa koji ne postoji, ključu se pridjeli defaultna vrijednost

[ ]:
```python
from collections import defaultdict
s = 'sayonara'
dd = defaultdict(int)
for letter in s:
    dd[letter] += 1

print(dd)
```

Usporedba s običnim dictionaryjem

[ ]:
```python
s = 'sayonara'
dd = dict()
for letter in s:
    if letter not in dd:      # prvo pojavljivanje slova
        dd[letter] = 0
    dd[letter] += 1

print(dd)

# kraće, pomoću get(kljuc, zadana_vrijednost)
dd = {}
for letter in s:
    dd[letter] = dd.get(letter, 0) + 1
print(dd)
```

`defaultdict(list)` - svaki novi ključ automatski dobije praznu listu

[ ]:
```python
from collections import defaultdict
groups = defaultdict(list)
for w in ['ana', 'banana', 'avion', 'breskva', 'cesta']:
    groups[w[0]].append(w)
print(dict(groups))
```

9.1.4 Counter

`collections.Counter` broji pojavljivanja elemenata (multiskup).

[ ]:
```
from collections import Counter
c = Counter('sayonara')
print(c)
print(c['a'], c['x'])      # nepostojeci kljuc daje 0
print(c.most_common(2))    # dva najcesca elementa
```

9.2 Set (skup)

- neuređena kolekcija elemenata u kojoj se svaki element pojavljuje samo jednom
- dvije varijante: `set` (**mutabilna**) i `frozenset` (**nemutabilna**)
- metode: `add`, `remove`, `copy`, `clear`
- operacije nad setovima:`intersection`, `union`, `difference`, `symmetric_difference`, `issuperset` i `issubset`

[ ]:
```
s1 = {1,2}
s2 = set([2,3,4])
s1.intersection(s2)
```

[ ]:
```
s = set("ana")
s.add('n')
s
```

[ ]:
```
s = frozenset(s)
s
```

[ ]:
```
#s.add('b')
#error
```

Set/frozenset mogu se napraviti iz bilo kojeg *iterable* objekta

[ ]:
```
lst = [1, 1, 2, 4, 5, 5]
s1 = set(lst)
print(s1)
s2 = frozenset(lst)
print(s2)
```

Npr. iz ključeva dictionaryja

[ ]:
```
d = {1:'aaa', 2:'bbb'}
s = set(d)
s
```

9.3 Rad sa kontejnerima

9.3.1 Deep copy vs. shallow copy

Pridruživanje operatorom `=` ne kopira objekt, nego mu daje još jedno ime

[ ]:
```
lst1 = [1,2,3]
lst2 = lst1
print(id(lst1), id(lst2))
```

[ ]:
```
lst1.append(4)
print(lst1, lst2)
```

9.3.2 Modul copy

[ ]:
```
import copy
lst1 = [1,2,3]
lst2 = copy.copy(lst1)
print(id(lst1), id(lst2))
```

Ako nema ugniježđenih elemenata, nemamo problem sa **shallow copy**

[ ]:
```
lst1.append(4)
print(lst1, lst2)
```

Ako ima ugniježđenih elemenata, imamo problem sa shallow copy

[ ]:
```
lst1 = [[1,1],[2,2],[3,3]]
lst2 = copy.copy(lst1)
print(id(lst1), lst1)
print(id(lst2), lst2)
```

[ ]:
```
lst1[0][0] = 42
print(id(lst1), lst1)
print(id(lst2), lst2)
```

Liste su različite ali sadrže reference na ugniježđene elemente.

[ ]:
```
lst1.append(3)
print(id(lst1), lst1)
print(id(lst2), lst2)
```

9.3.3 deep copy

[ ]:
```
lst1 = [[1,1],[2,2],[3,3]]
lst2 = copy.deepcopy(lst1)
print(id(lst1), lst1)
print(id(lst2), lst2)
```

[ ]:
```
lst1[0][0] = 42
print(id(lst1), lst1)
print(id(lst2), lst2)
```

Što je s nemutabilnim tipovima (tuple, string, frozenset)?

[ ]:
```
s = (1,2,3)
s2 = copy.copy(s)
```

[ ]:
```
id(s), id(s2)
```

[ ]:
```
s3 = copy.deepcopy(s)
```

[ ]:
```
id(s), id(s3)
```

Tuple, stringovi i frozenset su nemutabilni pa `copy` vraća **isti** objekt (isti `id`) — nema potrebe za kopijom jer se objekt ionako ne može promijeniti.

10 List (set, map) comprehension

```
new_list = [expression(i) for i in old_list if filter(i)]
```

[ ]:
```
word_list = ['ovo', 'je', 'lista', 'rijeci']
```

[ ]:
```
new_list = [word[0] for word in word_list]
print(new_list)
```

[ ]:
```
new_list = [word[-1] for word in word_list]
print(new_list)
```

[ ]:
```
word = 'lista'
idx = [i for i in range(len(word_list)) if word_list[i] == word]
print(idx)

# isto, citljivije pomocu enumerate
pairs = [(i, w) for i, w in enumerate(word_list) if w == word]
print(pairs)
```

Primjer

[ ]:
```
def myprint(i, x):
    print(i , "->", x)

lst = [[n**k for n in range(1, 11)] for k in range(10)]

print(lst)
```

[ ]:
```
# comprehension NE koristimo samo zbog nuspojava (npr. ispisa) - za to je␣
 ↪obicna petlja
for i, row in enumerate(lst):
    myprint(i, row)
```

[ ]:
```
for i, w in enumerate(word_list):
    print(i, w)
```

Primjer: primes

```
[ ]: m = 100
     n = 120

     # dovoljno je provjeriti djelitelje do korijena broja
     #list
     primes = [x for x in range(m, n) if all(x % y for y in range(2, int(x**0.5) +␣
      ↪1))]
     print(primes)

     #set
     primes = {x for x in range(m, n) if all(x % y for y in range(2, int(x**0.5) +␣
      ↪1))}
     print(primes)

     #generator
     primes = (x for x in range(m, n) if all(x % y for y in range(2, int(x**0.5) +␣
      ↪1)))
     print([p for p in primes])

     #tuple
     primes = tuple(x for x in range(m, n) if all(x % y for y in range(2, int(x**0.
      ↪5) + 1)))
     print(primes)
```

11 Matrice (liste lista)

Python nema ugrađeni tip za matricu; najjednostavnije je koristiti **listu lista** - svaki redak je lista.

```
[ ]: A = [[1, 2, 3],
          [4, 5, 6]]
     print(A[1][2])          # redak 1, stupac 2 -> 6
     print(len(A), len(A[0]))  # broj redaka i stupaca
```

Kreiranje matrice `m x n` pomoću list comprehensiona:

```
[ ]: m, n = 3, 4
     Z = [[0] * n for _ in range(m)]
     Z[0][0] = 1
     print(Z)
```

Zamka: `[[0] * n] * m` ne stvara `m` različitih redaka, nego `m` imena za **isti** redak!

```
[ ]: bad = [[0] * 3] * 3
     bad[0][0] = 1
     print(bad)                          # promijenili smo 'sva tri' retka
     print(id(bad[0]), id(bad[1]))       # isti objekt
```

Prolazak kroz matricu i ispis:

```
[ ]: A = [[1, 2, 3],
          [4, 5, 6],
          [7, 8, 9]]

     for row in A:
         for x in row:
             print(f'{x:3}', end='')
         print()

     # suma svakog retka i ukupna suma
     print([sum(row) for row in A])
     print(sum(sum(row) for row in A))
```

```
[ ]: # transponiranje pomocu zip i raspakiravanja
     AT = [list(col) for col in zip(*A)]
     print(AT)

     # jedinicna matrica
     I = [[1 if i == j else 0 for j in range(3)] for i in range(3)]
     print(I)
```

```
[ ]: # mnozenje matrica
     def mat_mul(A, B):
         m, k, n = len(A), len(B), len(B[0])
         return [[sum(A[i][t] * B[t][j] for t in range(k)) for j in range(n)] for i␣
      ↪in range(m)]

     print(mat_mul(A, I))
```

12 Još containera

- `Counter`
- `deque`
- `ChainMap`
- `namedtuple()`
- …

https://docs.python.org/3/library/collections.html

12.1 `deque` - red s dva kraja

- `collections.deque` omogućuje brzo dodavanje i uklanjanje s **oba** kraja.
- `lst.pop(0)` na listi je spor (pomiče sve elemente), a `deque.popleft()` je brz.
- Tipična upotreba: **red** (*queue*, FIFO).

[ ]:
```
from collections import deque

q = deque()
q.append('a')          # dodaj na kraj
q.append('b')
q.append('c')
print(q.popleft())     # uzmi s pocetka -> 'a'
q.appendleft('z')      # dodaj na pocetak
print(q)
```

[ ]:
```
# obrada elemenata redom kojim su dodani, dok red nije prazan
q = deque([1])
while q:
    x = q.popleft()
    print(x, end=' ')
    if x < 20:
        q.append(2 * x)
```

12.2 heapq - red s prioritetom

- Modul heapq koristi običnu listu kao **min-heap**: heappop uvijek vraća **najmanji** element.
- Ubacivanje i vađenje su brzi (logaritamska složenost).
- Za parove (prioritet, podatak) koristimo n-torke - uspoređuju se po prvom elementu.

[ ]:
```
import heapq

h = []
heapq.heappush(h, 5)
heapq.heappush(h, 1)
heapq.heappush(h, 3)
print(heapq.heappop(h))    # 1
print(heapq.heappop(h))    # 3
```

[ ]:
```
# zadaci s prioritetom
tasks = []
heapq.heappush(tasks, (3, 'citati'))
heapq.heappush(tasks, (1, 'predati zadacu'))
heapq.heappush(tasks, (2, 'vjezbati'))

while tasks:
    priority, task = heapq.heappop(tasks)
    print(priority, task)
```

12.3 Beskonačnost: float('inf')

Korisno kao početna vrijednost kad tražimo minimum.

[ ]:
```
inf = float('inf')
print(inf > 10**100, -inf < -10**100)

numbers = [17, 4, 42, 8]
best = inf
for x in numbers:
    if x < best:
        best = x
print(best)
```

13 Funkcije

[ ]:
```
def f(n, s, l):
    n = 2
    s += "novi"
    l.extend([5,6])
    print('f:', n, s, l)

def main():
    n = 1
    s = "test"
    l = [0,1,2,3]
    print('main:', n, s, l)
    f(n, s, l)
    print('main: ', n, s, l)

main()
```

call-by-value ili call-by-reference?

- nijedno od toga

call-by-sharing (call-by-object, call-by-object-reference)

- argumenti su objekti i mogu se mijenjati samo ako su mutabilni
- prilikom izvršavanja funkcije objekt dobije drugo ime
- https://docs.python.org/3/faq/programming.html#how-do-i-write-a-function-with-output-parameters-call-by-reference

[ ]:
```
def funkcija(n):
    print(id(n), n)
    n+=2
    print(id(n), n)

k = 3
print(id(k), k)
funkcija(k)
print(id(k), k)
```

[ ]:
```
def funkcija(n):
    print(id(n), n)
    n+=[2]
    print(id(n), n)

k = [3]
print(id(k), k)
funkcija(k)
print(id(k), k)
```

Document string

[ ]:
```
def is_prime(n):
    """
    check if n is prime
    """
    return n > 1 and all(n % k for k in range(2, int(n**0.5) + 1))

help(is_prime)
```

Argumenti funkcije

- *required arguments*

[ ]:
```
def fun(n, k):
    return n**k

#fun(3)     #error
```

- *keyword/named arguments*

[ ]:
```
fun(k = 5, n = 2)
```

- *default arguments*

[ ]:
```
def fun(n, k = 1):
    return n**k

fun(4)
```

Varijabilni broj argumenata

[ ]:
```
def print_val(*lst):
    for val in lst:
        print(val, end=' ')
    print("\n")


print_val(2,3)
print_val(2,3,4)
```

```
print_val(5,6,'a', 8,8,9)
```

Argumenti funkcije

- *variable-length argument* (pakiranje argumenata)

[ ]:
```
def fun(n, *args):
    print("args are ", type(args), args)
    ret = []
    for i in args:
        ret.append(n**i)
    return ret

fun(2, 7, 8, 9)
```

ali ne može ovako

[ ]:
```
lst = [7, 8, 9]
# fun(2, lst)          # TypeError: lista se ne raspakira sama
```

Tome možemo doskočiti tako da raspakiramo argumente pri pozivu:

[ ]:
```
fun(2, *lst)
```

Argumenti funkcije

- *variable-length argument* (pakiranje *keyword* argumenata)

[ ]:
```
def fun2(n, **kwargs):
    print("kwargs are ", type(kwargs), kwargs)
    ret = []
    for i in kwargs.values():
        ret.append(n**i)
    return ret

print(fun2(2, a = 2, b = 3))
```

opet ne može ovako

[ ]:
```
dct = {'a':2, 'b':3}
# print(fun2(2, dct))     # error
```

ali može ovako (raspakiravanje dictionaryja)

[ ]:
```
dct = {'a':2, 'b':3}
print(fun2(2, **dct))
```

13.1 Rekurzija

Funkcija može pozivati samu sebe. Potreban je **bazni slučaj** koji zaustavlja rekurziju.

[ ]:
```python
def factorial(n):
    if n <= 1:          # bazni slucaj
        return 1
    return n * factorial(n - 1)

print(factorial(5))
```

[ ]:
```python
# rekurzivni zbroj ugnijezdjenih lista
def nested_sum(lst):
    total = 0
    for x in lst:
        if isinstance(x, list):
            total += nested_sum(x)
        else:
            total += x
    return total

print(nested_sum([1, [2, 3], [4, [5, 6]]]))
```

Python ograničava dubinu rekurzije (zadano 1000 poziva; Jupyter tu granicu obično podiže, npr. na 3000). Za dublju rekurziju granica se može povećati:

[ ]:
```python
import sys
print(sys.getrecursionlimit())
# sys.setrecursionlimit(10000)
```

13.2 Built-in funkcije

- `len` - ulaz je sekvenca (list, string) ili kolekcija (dictionary, set), vraća duljinu
- `sum` - ulaz je iterirajuća kolekcija, vraća sumu
- `sorted` - vraća novu sortiranu listu (metoda `sort` mijenja postojeću listu)
- `enumerate` - vraća enumerirani objekt, koristi se za iteriranje `enumerate(iterable, start=0)`
- …

[ ]:
```python
lst=['a','b','c']
for el in enumerate(lst):
    print(el)
```

https://docs.python.org/3/library/functions.html

14 I/O

standardni ulaz i izlaz

- `print` je funkcija: `print(expr1, expr2)`
- `input()` uvijek vraća **string**
- za broj koristimo pretvorbu: `int(input())`, `float(input())`
- (u Pythonu 2 `input()` je evaluirao unos - u Pythonu 3 to više nije tako)

```
[ ]: a = input()
```

```
[ ]: # a + 2          # TypeError: string + int
```

```
[ ]: a + '2'           #ok
```

```
[ ]: a = int(input())
```

```
[ ]: a + 1
```

15 Rad s datotekama

File objekt

- kreira se sa `open()`
- čitanje blok po blok ili liniju po liniju
- pisanje
- seek
- flush
- zatvaranje s `close` ili garbage collector

Otvaranje datoteke `file_object = open(name [, mode])`

- `name` je ime datoteke
- `mode` je mod otvaranja (`r`, `w`, `a`, `r+`, `w+`, `b`)
- `file_object` je iterabilan

```
[ ]: f = open("primjer.txt")
```

```
[ ]: f
```

```
[ ]: f.name
```

```
[ ]: f.mode
```

```
[ ]: f.close()
```

Otvaranje datoteke

```
[ ]: with open('primjer.txt', 'r') as f:
         for line in f:
             print(line, end='')
```

- Naredba `with` koristi *context manager*: poziva se metoda `__enter__`, izvršavaju se naredbe, na kraju se poziva metoda `__exit__` (zatvaranje datoteke)
- Metoda `__exit__` se poziva i ako se dogodila iznimka prilikom izvođenja naredbi.

Lokacija datoteke

- Kad se navodi samo ime datoteke, ona se traži u tekućem direktoriju (engl. *working directory*).
- Lokacija datoteke može biti relativna u odnosu na tekući direktorij ili apsolutna.

- OS-X, Linux, Unix: delimiter je `'/'`
- Windows: delimiter je `'\'`
- koristiti *raw string* `r'C:\put\do\datoteke'` ili `'\\'`

Čitanje iz datoteke

`block = f.read(n)` - čita najviše `n` znakova (u binarnom načinu bajtova), vraća string

`block = f.read()` - čita čitavu datoteku, vraća string

`line = f.readline()` - čita cijelu liniju, vraća string

`line = f.readline(n)` - čita najviše `n` znakova tekuće linije

`line = f.readlines()` - čita sve linije, vraća listu stringova

`line = f.readlines(sizehint)` - čita `sizehint` znakova, pa do kraja tekuće linije

[ ]:
```
f = open('primjer.txt')
while True:
    line = f.readline()
    if line == '':
        break
    print(line, end='')
f.close()
```

Čitanje iz datoteke

- Za manje datoteke dobre su `read` funkcije.
- Za veće datoteke dobro je koristiti iteriranje po file generatoru.

[ ]:
```
with open('primjer.txt') as f:
    for line in f:
        print(line, end='')
```

Pisanje u datoteku

`f.write(s)`

`print(s, file=f)`

- piše string u datoteku
- mora primiti string
- pisanje se baferira, `flush` ili `close` će napraviti operaciju vidljivom

Operacije s datotekama

- prilikom čitanja datoteke dobije se string
- za pisanje u datoteku također nam je potreban string

15.1 Čitanje i pisanje `csv` datoteka

- pisanje

[ ]:
```
import csv
rows = [['ime', 'bodovi'], ['Ana', 92], ['Marko', 45]]
with open('output.csv', 'w', newline='') as f:
    writer = csv.writer(f)
    writer.writerows(rows)
```

- čitanje

[ ]:
```
import csv
with open('output.csv') as f:
    reader = csv.reader(f, delimiter=',')
    for row in reader:
        print(row)       # svaki redak je lista stringova
```

https://docs.python.org/3/library/csv.html

16 Iznimke

Exceptions

- kada se dogodi *runtime error*, kreira se **iznimka** (engl.*exception*), program se zaustavlja i ispisuje se poruka o greški

Primjeri iznimaka (za izvršavanje ukloniti #):

[ ]:
```
# 22/0                                          # ZeroDivisionError
```

[ ]:
```
l = []
# l[2]                                          # IndexError
```

[ ]:
```
dct = {}
# dct['kljuc']                                  # KeyError
```

- koristeći `try` i `except` naredbe možemo rukovati iznimkom

[ ]:
```
filename = input('unesi ime datoteke ')
try:
    f = open(filename, 'r')
except FileNotFoundError:
    print('nema te datoteke')
```

Exceptions - raise

Možemo i sami podići iznimku

[ ]:
```
def input_even():
    n = int(input("Unesi parni broj: "))

    if n % 2 != 0:
        raise ValueError('Broj nije paran')
```

```
    result = n + 'aaa'  # TypeError
    return result

def main():
    try:
        n = input_even()
        print(f"Rezultat: {n}")
    except ValueError as err:
        print(f"Pogrešan unos: {err}")
    except TypeError as err:
        print(f"Greška u tipu podataka: {err}")
    except Exception as err:
        print(f"Nepoznata greška: {type(err).__name__}: {err}")

    print('Nastavljamo program')

if __name__ == '__main__':
    main()
```

Exceptions - finally

- `else` se izvršava ako nije došlo do iznimke
- `finally` se izvršava bez obzira da li se dogodila iznimka

[ ]:
```
def concat(a, b):
    try:
        res = a + b
        c = a / b
    except TypeError:
        print('Greška: Ne mogu spojiti ove tipove podataka')
    except ZeroDivisionError:
        print('Greška: Dijeljenje s nulom!')
    else:
        print(f'Rezultat: {res}')
    finally:
        print('Operacija završena!')

# Testiraj različite scenarije
print('1. Spajamo brojeve:')
concat(5, 2)

print('\n2. Spajamo string i broj:')
concat('a', 4)

print('\n3. Dijeljenje s nulom:')
concat(5, 0)
```

```
[ ]: print('neuhvacena iznimka')
     #concat(5, 0)      # error
```

17 Moduli

- Svaka `*.py` datoteka je modul. Modul je *namespace*.
- Moduli mogu biti built-in moduli, third party moduli ili lokalni moduli (moduli projekta).
- Više modula zajedno čini paket (*package*). Popis instaliranih paketa: `pip list` ili `pip freeze`.
- Učitavanje modula naredbom `import`. Izvođenje naredbi modula obavlja se samo jednom.

```
[ ]: import math

     n = int(input('unesi broj '))
     print('broj znamenki je ', int(math.log10(n)) + 1)
```

Načini uvoženja:

- `import module`
  - korištenje: `module.foo`
- `import module as mo`
  - korištenje: `mo.foo`
- `from module import foo`
  - korištenje: `foo`
  - gubi se kontekst, npr. je li funkcija `floor()` ustvari `math.floor()`
- `from module import *`
  - puni sve javne objekte modula u prostor imena
  - **ne preporuča se** (ne vidi se odakle dolazi koje ime, a imena se mogu preklopiti)

https://docs.python-guide.org/writing/structure/

Varijabla `__name__` je postavljena na `__main__` kada se izvodi skripta ili pokreće interaktivni prompt. Kad se modul uvozi naredbom `import`, `__name__` je ime modula.

```
[ ]: if __name__ == '__main__':
         print("samostalno")
     else:
         print("importano")
```

18 Prostor imena

Namespace

Namespace ili **prostor imena** je sistem po kojem se kreiraju jedinstvena imena da bi se izbjegla neodređenost (primjerice ista imena funkcija u različitim modulima).

Doseg varijabli

- imena nastaju kad im se prvi put dodijeli vrijednost
- s obzirom na mjesto gdje se dodjeljivanje izvodi, povezuje se s odgovarajućim namespaceom

- unutar funkcije imena su lokalna
- svaki poziv funkcije stvara novi lokalni doseg
- unutar funkcije moguće je deklarirati ime kao globalno
- globalni doseg je unutar modula

[ ]:
```
print("modul name ", __name__)

n = 20

def fcja1():
    n = 15
    print(n, locals())

def fcja2():
    print(n)     # UnboundLocalError: n je lokalna jer joj se kasnije␣
 ↪pridjeljuje vrijednost
    n = 15

fcja1()
#fcja2()                    #error
```

Python smatra `n` lokalnom ukoliko je unutar funkcije, ako želimo da je smatra globalnom, treba to naglasiti.

[ ]:
```
def fcja2():
    global n
    print(n, locals())
    n = 15
    print(n, locals())

fcja2()
```

19 Klase i objekti

Objekt je najmanja jedinica podataka koja se može izmjenjivati između dijelova programa

- smještati u varijable,
- slati u i iz funkcije,
- spremati u složenije strukture (liste, rječnike).

Svaki objekt ima svojstva: id, tip (type), vrijednost (state)

Klase - **Tip** je kategorija koja definira svojstva zajednička objektima: metode i atribute. - Objekt danog tipa je instanca tipa. - Svaki objekt zna svoj tip `type(obj)` - strongly typed language - Ugrađeni tipovi su numerički tipovi, stringovi, bool,.. - Novi tip nastaje kreiranjem python klase.

`class ImeKlase(Base1, Base2, ...):    DataAttributes    ClassMethods`

- definicija klase
- ako se klasa izvodi iz baznih klasa, ona sadrži njene atribute (koji se kasnije mogu pregaziti)

- naredbe unutar definicije klase predstavljaju implementaciju klase
- instance (objekti) se kreiraju prilikom poziva klase

Primjer klase

[ ]:
```
class Point:
    pass

p = Point()
p.x = 2
p.y = 3

print(f'({p.x:.2f}, {p.y:.2f})')
```

Metode klase

[ ]:
```
import math
class Point:

    def distance(self, x = 0, y = 0):
        dx = (self.x - x) ** 2
        dy = (self.y - y) ** 2
        return math.sqrt(dx + dy)

    def move(self, x, y = 0):
        self.x += x
        self.y += y

p = Point()
p.x = 1
p.y = 1

print(f'({p.x:.2f}, {p.y:.2f})')
print(p.distance())
p.move(2)
print(f'({p.x:.2f}, {p.y:.2f})')
```

Konstruktor

[ ]:
```
class Point:
    def __init__(self, x = 0, y = 0):
        self.x = x
        self.y = y

p = Point(5, 3.5)

print(f'({p.x:.2f}, {p.y:.2f})')
```

Metoda `__str__`

[ ]:
```python
class Point:
    def __init__(self, x = 0, y = 0):
        self.x = x
        self.y = y
    def __str__(self):
        return f'({self.x:.2f}, {self.y:.2f})'

p = Point(2, 3)
print(p)
```

Preopterećenje operatora

- dunder methods - **d**ouble **under**score methods
- https://docs.python.org/3/reference/datamodel.html#special-method-names

[ ]:
```python
class Point:
    def __init__(self, x = 0, y = 0):
        self.x = x
        self.y = y
    def __str__(self):
        return f'({self.x:.2f}, {self.y:.2f})'
    def __add__(self, other):
        return Point(self.x + other.x, self.y + other.y)

p1 = Point(2, 3)
p2 = Point(1, 1)
p3 = p1 + p2
print(p3)
```

Python nema `private`; po konvenciji ime koje počinje s _ nije za vanjsku upotrebu.

20 Python - funkcionalne karakteristike jezika

Programske paradigme u Pythonu

- proceduralno (funkcija, iteracija, uvjetne naredbe)
- objektno orijentirano (klase, objekti, metode)
- funkcionalno (**stateless**, čiste funkcije, nepromjenjivi podaci)

Funkcije u Pythonu

- funkcije su first-class citizens
- funkcije mogu biti argumenti druge funkcije
- povratna vrijednost funkcije može biti funkcija

Funkcija kao argument

[ ]:
```python
def calc(f, a, b):
     return f(a, b)
```

```python
def add(a, b):
     return a + b

def multi(a, b):
     return a * b

print(calc(add, 5, 6))
print(calc(multi, 5, 6))
```

20.1 Anonimne funkcije (lambda)

[ ]:
```python
def calc(f, a, b):
     return f(a, b)

print(calc(lambda a, b: a + b, 5, 6))
print(calc(lambda a, b: a * b, 5, 6))
```

[ ]:
```python
def apply_operation(operation, a, b):
    """Primjenjuje operaciju na dva broja"""
    return operation(a, b)

operations = {
    'add': lambda x, y: x + y,
    'subtract': lambda x, y: x - y,
    'multiply': lambda x, y: x * y,
    'divide': lambda x, y: x / y if y != 0 else None
}

for op_name, op_func in operations.items():
    result = apply_operation(op_func, 10, 5)
    print(f"10 {op_name} 5 = {result}")
```

20.2 map

Primjenjuje funkciju na svaki element. Primjer: svaki element pomnožiti s dva.

[ ]:
```python
def double(a):
    return a * 2

lst = [1, 2, 3, 4]
res = []
for el in lst:
    res.append(double(el))
print(res)
```

- Koristeći `map` (vraća iterator, pa se za ispis pretvara u listu):

```
[ ]: lst = [1, 2, 3, 4]
     print(list(map(double, lst)))
     print(list(map(lambda a: a * 2, lst)))
```

20.3 filter

Zadržava samo one elemente koji zadovoljavaju neki uvjet.

```
[ ]: def filter_iter(elems):
         res = []
         for el in elems:
             if el % 2 == 0:
                 res.append(el)
         return res

     lst = [1,2,3,4]
     lst = filter_iter(lst)
     print(lst)
```

- Koristeći `filter`:

```
[ ]: lst = [1, 2, 3, 4]
     print(list(filter(lambda a: a % 2 == 0, lst)))
```

Isto se u praksi češće piše pomoću *list comprehensiona*:

```
[ ]: lst = [1, 2, 3, 4]
     print([a * 2 for a in lst])              # kao map
     print([a for a in lst if a % 2 == 0])    # kao filter
```

20.4 reduce

`reduce` sažima sekvencu u jednu vrijednost: funkcija se primjenjuje na akumulator i sljedeći element, redom, dok se elementi ne potroše.

Sintaksa: `reduce(funkcija, iterabil, početna_vrijednost)`

```
[ ]: from functools import reduce

     lst = [1, 2, 3, 4]
     res = reduce(lambda accum, el: accum + el, lst, 0)
     print(res)
```

Korak po korak za `reduce(lambda accum, el: accum + el, [1, 2, 3, 4], 0)`:

korak	`accum`	`el`	novi `accum`
1	0	1	1
2	1	2	3
3	3	3	6
4	6	4	10

korak	`accum`	`el`	novi `accum`

Najčešće redukcije već postoje kao ugrađene funkcije: `sum`, `min`, `max`, `any`, `all` (i `''.join` za stringove). U praksi se koriste one, a `reduce` kad odgovarajuće ugrađene funkcije nema:

[ ]:
```
lst = [1, 2, 3, 4]
print(reduce(lambda accum, el: accum * el, lst, 1))     # produkt

sets = [{1, 2}, {2, 3}, {5}]
print(reduce(lambda a, b: a | b, sets, set()))           # unija svih skupova
```

Map - filter - reduce zajedno: odabrati parne elemente (filter), kvadrirati ih (map) i zbrojiti (reduce).

[ ]:
```
lst = [1, 2, 3, 4, 5, 6]
print(reduce(lambda a, b: a + b, map(lambda x: x * x, filter(lambda x: x % 2 ==␣
 ↪0, lst)), 0))

# isto, idiomatski Python
print(sum(x * x for x in lst if x % 2 == 0))
```

20.5 Generatori

- Funkcija koja vrijednosti vraća s `yield` (umjesto `return`) je **generator**.
- Vrijednosti se ne spremaju sve u memoriju, nego se generiraju jedna po jedna, tek kad su potrebne (*lazy evaluation*).
- Po generatoru se može iterirati **samo jednom**.

[ ]:
```
def squares(n):
    for i in range(n):
        yield i ** 2

gen = squares(5)
print(type(gen))

for x in gen:
    print(x, end=' ')
print()

print(list(gen))    # prazna lista - generator je vec potrosen
```